

Inheritance

Abdullah Al Asif, Lecturer, CSE, BAUST
S. M Golam Rifat, Lecturer, CSE, BAUST

Java Inheritance

- A feature of OOP, that allows to create a new class from an existing class.
- The new class is called **subclass**.
- The existing class from where child class is derived is called **superclass**.
- To inherit a class, **extends** keyword is used.
- Multilevel inheritance but no multiple inheritance.
- There is a special way to call the superclass's constructor.
- There is automatic dynamic method dispatch (runtime polymorphism).
- Inheritance provides code reusability (code of any class can be used by extending that class).

Simple Inheritance

```
class Person { 3 usages 1 inheritor
    String name; 4 usages
    int age; 4 usages

    void showPerson() { 2 usages
        System.out.println(name + " " + age);
    }
}

class Student extends Person { 2 usages
    int studentId; 3 usages

    void showStudent() { 1 usage
        System.out.println(studentId);
    }

    void showAll() { 1 usage
        System.out.println(name + " " + age + " " + studentId);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Person personObj = new Person();
        personObj.name = "Rahim";
        personObj.age = 40;
        personObj.showPerson();

        Student studentObj = new Student();
        studentObj.name = "Karim";
        studentObj.age = 20;
        studentObj.studentId = 101;

        studentObj.showPerson();
        studentObj.showStudent();
        studentObj.showAll();
    }
}
```

Inheritance and Member Access

```
class Person { 1 usage 1 inheritor
    String name; 2 usages
    private int age; 2 usages

    void setAge(int a) { 1 usage
        age = a;
    }
}

class Student extends Person { 2 usages
    int studentId; 1 usage

    void showStudent() { 1 usage
        System.out.println(name);
        System.out.println(age); //Error: age is not accessible here
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Student studentObj = new Student();

        studentObj.name = "Rahim";
        studentObj.setAge(20);
        studentObj.studentId = 101;

        studentObj.showStudent();
    }
}
```

- A class member that has been declared as private will remain private to its class.
- It is not accessible by any code outside its class, including subclasses.

Superclass variable reference to Subclass object (Upcasting)

```
class Person { 4 usages 1 inheritor
    String name; 3 usages
    int age; 3 usages

    void showPerson() { 2 usages
        System.out.println(name + " " + age);
    }
}

class Student extends Person { 2 usages
    int studentId; 3 usages

    Student(String n, int a, int id) { 1 usage
        name = n;
        age = a;
        studentId = id;
    }

    void showStudent() { no usages
        System.out.println(studentId);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Student studentObj = new Student( n: "Rahim", a: 20, id: 101);

        System.out.println(studentObj.studentId); // OK
        studentObj.showPerson(); // OK

        Person personObj = studentObj; // Person reference refers to Student object

        System.out.println(personObj.name); // OK
        System.out.println(personObj.age); // OK
        personObj.showPerson(); // OK

        System.out.println(personObj.studentId); // Error: studentId is not defined in Person

        Person person = new Person();

        Student student = person; // Error: cannot assign Person reference to Student
    }
}
```

Superclass variable reference to Subclass object (Upcasting)

```
public class Main {  
    public static void main(String[] args) {  
  
        Student studentObj = new Student(n: "Rahim", a: 20, id: 101);  
  
        System.out.println(studentObj.studentId); // OK  
        studentObj.showPerson(); // OK  
  
        Person personObj = studentObj; // Person reference refers to Student object  
  
        System.out.println(personObj.name); // OK  
        System.out.println(personObj.age); // OK  
        System.out.println(personObj.studentId); // We can't access extra property  
        personObj.showPerson(); // OK  
  
        System.out.println(personObj.studentId); // Error: studentId is not defined in Person  
  
        Person person = new Person();  
  
        Student student = (Student) person; // Error: cannot assign Person reference to Student  
        System.out.println(student.studentId); // this will produce runtime error  
    }  
}
```

Using super to call Superclass Constructors

```
class Person { 2 usages 1 inheritor
    String name; 6 usages
    int age; 6 usages

    Person(Person p) { 1 usage
        name = p.name;
        age = p.age;
    }

    Person(String n, int a) { 1 usage
        name = n;
        age = a;
    }

    Person() { 1 usage
        name = "Unknown";
        age = 0;
    }

    Person(String n) { 1 usage
        name = n;
        age = 0;
    }
}
```

```
class Student extends Person { 11 usages
    int studentId; 6 usages

    Student(Student s) { 1 usage
        super(s);
        studentId = s.studentId;
    }

    Student(String n, int a, int id) { 2 usages
        super(n, a);
        studentId = id;
    }

    Student() { 1 usage
        super();
        studentId = 0;
    }

    Student(String n, int id) { 1 usage
        super(n);
        studentId = id;
    }

    void print() { 5 usages
        System.out.println("Name: " + name + ", Age: " + age + ", ID: " + studentId);
    }
}
```

Super() must always be the first statement executed inside a subclass' constructor

Using super to call Superclass Constructors

```
public class Main {  
    public static void main(String[] args) {  
  
        Student student1 = new Student(n: "Rahim", a: 20, id: 101);  
        Student student2 = new Student(n: "Karim", a: 21, id: 102);  
        Student student3 = new Student();  
        Student student4 = new Student(n: "Hasan", id: 103);  
        Student student5 = new Student(student1);  
  
        student1.print();  
        student2.print();  
        student3.print();  
        student4.print();  
        student5.print();  
    }  
}
```

Using super to access Superclass hidden members

```
class Person { 1 usage 1 inheritor
    String name; 3 usages
    int age; 2 usages

    Person(String name, int age) { 1 usage
        this.name = name;
        this.age = age;
    }

    void show() { 2 usages
        System.out.println("Person name: " + name);
        System.out.println("Age: " + age);
    }
}
```

```
class Student extends Person { 2 usages
    String name; // this name hides the name in Person 2 usages
    int studentId; 2 usages

    Student(String personName, int age, String studentName, int id) { 1 usage
        super(personName, age);

        this.name = studentName;
        studentId = id;
    }

    void show() { 2 usages
        System.out.println("Name in superclass: " + super.name);
        System.out.println("Name in subclass: " + name);

        super.show();

        System.out.println("Student ID: " + studentId);
    }
}
```

Using super to access Superclass hidden members

```
public class Main {  
    public static void main(String[] args) {  
  
        Student studentObj = new Student( personName: "Rahim", age: 20, studentName: "Karim", id: 101);  
  
        studentObj.show();  
    }  
}
```

Multilevel Inheritance

```
class Person { 1 usage 2 inheritors
    String name; 1 usage
    int age; 1 usage

    Person() { 1 usage
        System.out.println("Inside Person's constructor");
    }
}

class Student extends Person { 1 usage 1 inheritor
    int studentId; 1 usage

    Student() { 1 usage
        System.out.println("Inside Student's constructor");
    }
}
```

```
class GraduateStudent extends Student { 2 usages
    String researchTopic; 1 usage

    GraduateStudent() { 1 usage
        System.out.println("Inside GraduateStudent's constructor");
    }
}

public class Main {
    public static void main(String[] args) {

        GraduateStudent gs = new GraduateStudent();

        gs.name = "Rahim";
        gs.age = 24;
        gs.studentId = 101;
        gs.researchTopic = "Artificial Intelligence";
    }
}
```

Method Overriding

```
class Person { 1 usage 1 inheritor
```

```
    String name; 3 usages
```

```
    int age; 3 usages
```

```
    void show() { 1 usage 1 override
```

```
        System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
```

```
    }
```

```
}
```

```
class Student extends Person { 2 usages
```

```
    int studentId; 2 usages
```

```
    @Override 1 usage
```

```
    void show() {
```

```
        System.out.println("Name: " + name);
```

```
        System.out.println("Age: " + age);
```

```
        System.out.println("Student ID: " + studentId);
```

```
    }
```

```
}
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Student studentObj = new Student();
```

```
        studentObj.name = "Rahim";
```

```
        studentObj.age = 20;
```

```
        studentObj.studentId = 101;
```

```
        studentObj.show();
```

```
    }
```

```
}
```

Dynamic Method Dispatch

```
class Person { 3 usages 1 inheritor

    void show() { 2 usages 1 override
        System.out.println("Inside Person's show()");
    }
}

class Student extends Person { 1 usage

    @Override 2 usages
    void show() {
        System.out.println("Inside Student's show()");
    }
}

public class Main {

    public static void main(String[] args) {
        Person obj;
        obj = new Person();
        obj.show();
        obj = new Student();
        obj.show();
    }
}
```

Dynamic Method Dispatch is the mechanism by which an overridden method is called based on the actual object at runtime, not the reference type.

Abstract Class

```
abstract class Person { 3 usages 1 inheritor
```

```
String name; 2 usages
```

```
int age; 2 usages
```

```
abstract void displayRole(); 1 usage 1 implementation
```

```
void displayBasicInfo() { 2 usages
```

```
    System.out.println("Name: " + name);
```

```
    System.out.println("Age: " + age);
```

```
}
```

```
}
```

```
class Student extends Person { 2 usages
```

```
int studentId; 2 usages
```

```
Student(String name, int age, int studentId) { 1 usage
```

```
    this.name = name;
```

```
    this.age = age;
```

```
    this.studentId = studentId;
```

```
}
```

```
@Override 1 usage
```

```
void displayRole() {
```

```
    System.out.println("Role: Student");
```

```
}
```

```
void displayStudentInfo() { 1 usage
```

```
    displayBasicInfo();
```

```
    System.out.println("Student ID: " + studentId);
```

```
}
```

```
}
```

Abstract Class

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Person p = new Person(); // ERROR  
        Student s = new Student( name: "Mahir", age: 20, studentId: 101);  
        s.displayBasicInfo();  
        s.displayRole();  
        s.displayStudentInfo();  
    }  
}
```

We cannot directly create an object of Person as it is abstract

Anonymous Subclass

```
abstract class Person { 3 usages 2 inheritors

    String name; 3 usages
    int age; 3 usages
    abstract void displayRole(); no usages 2 imp

    void displayBasicInfo() { 2 usages
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
```

```
class Student extends Person { no usages

    int studentId; 2 usages

    Student(String name, int age, int studentId) { no usages
        this.name = name;
        this.age = age;
        this.studentId = studentId;
    }

    @Override no usages
    void displayRole() {
        System.out.println("Role: Student");
    }

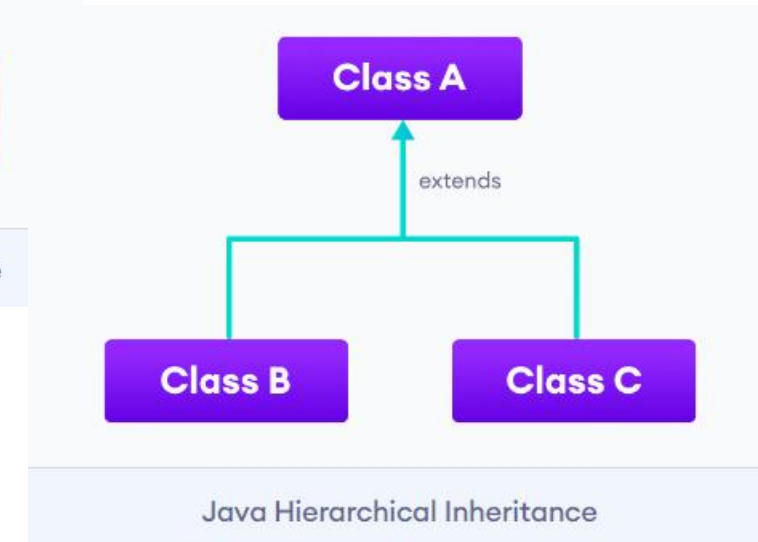
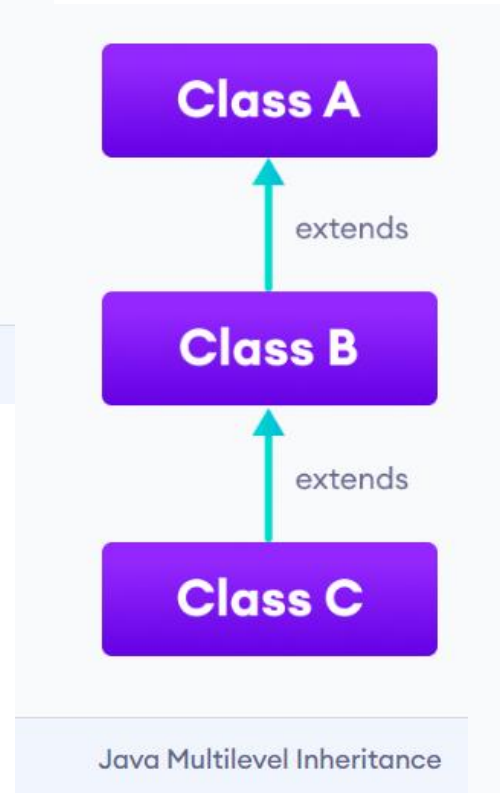
    void displayStudentInfo() { no usages
        displayBasicInfo();
        System.out.println("Student ID: " + studentId);
    }
}
```

Anonymous Subclass

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Person p1=new Person() {  
            @Override no usages  
            void displayRole() {  
                System.out.println("Role: Person");  
            }  
        };  
        p1.age=10;  
        p1.name="Karim";  
        p1.displayBasicInfo();  
  
    }  
}
```

Types of Inheritance

- **Single Inheritance:**
 - A single subclass extends from a single superclass.
- **Multilevel Inheritance:**
 - A subclass extends from a superclass and then the same subclass acts as a superclass for another class
- **Hierarchical Inheritance:**
 - multiple subclasses extend from a single superclass



Access Specifier

Access Modifiers In Java

Access Modifier	Within the Class	Other Classes [Within the Package]	In Subclasses [Within the package and other packages]	Any Class [In Other Packages]
public	Y	Y	Y	Y
protected	Y	Y	Y	N
default	Y	Y	Same Package – Y Other Packages – N	N
private	Y	N	N	N

Y – Accessible
N – Not Accessible

Using final with Inheritance

To Prevent Overriding

```
class Person { 1 usage 1 inheritor

    final void show() { no usages
        System.out.println("This is a final method in Person.");
    }
}

class Student extends Person { no usages

    void show() { // Error! Cannot override final method no usages
        System.out.println("Illegal!");
    }
}
```

Using final with Inheritance

To Prevent Inheritance

```
final class Person { 1 usage

    void show() { no usages
        System.out.println("Person");
    }
}

// Error! Cannot inherit from final Person
class Student extends Person { no usages

    void show() { no usages
        System.out.println("Student");
    }
}
```

Practice Problem 1

- Create two classes: a **Person** class and a **Student** class. The Student class should inherit from the Person class.
- Person class has two attributes **name** (string) and **age** (int), **displayDetails()** method to print the name and age of the person
- Student class inherits Person class, has a attribute named **studentId** (string) and a method **displayStudentId()** to print the studentId of the student.
- Set variable values from the main function.
- Display name, age from Person class.
- Display studentId from Student class.

Practice Problem 2

- Design three classes: **Animal**, **Mammal**, and **Dog**. The Mammal class should inherit from Animal, and the Dog class should inherit from Mammal.
- Animal Class:
 - Attribute: **species** (string)
 - Method: **displaySpecies()**: Prints the species of the animal.
- Mammal Class: Inherits from Animal
 - Attribute: **hasFur** (boolean)
 - Method: **displayMammalCharacteristics()**: Displays if the mammal has any fur.
- Dog Class: Inherits from Mammal
 - Attribute: **breed** (string)
 - Method: **displayBreed()**: Displays the breed of the dog.
- In the main method, create an instance of Dog, assign values to all its attributes using constructor, and call the methods to display the details.

References

- **Java - The Complete Reference by Herbert Schildt**
- <https://www.geeksforgeeks.org/java/>
- <https://www.w3schools.com/java/>
- <https://www.javatpoint.com/java-tutorial>
- <https://www.tutorialspoint.com/java/index.htm>
- <https://www.programiz.com/java-programming>
- **<https://github.com/Rafiquzzaman420/Free-Programming-Books>**